

UNIVERSIDADE DA CORUÑA

Programación de Sistemas

3.1 Clean Code, CLEAN Architecture y Git Flow

Carlos Vázquez Regueiro
carlos.vazquez.regueiro@udc.es

Índice

1. **Clean Code**
2. Principios SOLID
3. Clean architecture
4. Git y Git Flow

Clean Code - Introducción

> "The ratio of time spent reading versus writing is well over 10 to 1. We are constantly reading old code as part of the effort to write new code. ...[Therefore,] making it easy to read makes it easier to write."
>
> -- *Robert C. Martin*

- Cualquier código de más de una semana es heredado
- El código es como el pescado, en una semana huele mal
- El código heredado huele mal
- El código heredado es una oportunidad de mejora
- Reescribirlo para entenderlo es una necesidad



Recomendaciones - Nombres

> “There are only two hard things in Computer Science: cache invalidation and naming things”
> -- Phil Karlton

- Dar sentido mediante los nombres
- Claridad
- Explicar la **intención**. Por qué se hace
- Reglas
- Sustantivos para variables / propiedades

Recomendaciones - Vocabulario

Vocabulario de **entidades y propiedades** de negocio

- client, customer
- document, file

Verbos para funciones / métodos

Vocabulario para **relaciones y acciones** comunes

- is, has, get, set, can, must, do
- create, read, update, delete
- select, insert, update, delete
- get, post, put, delete

Recomendaciones - Consistencia

> "A long descriptive name is better than a short enigmatic name. A long descriptive name is better than a long descriptive comment. You should name a variable using the same care with which you name a first-born child"
> -- Robert C. Martin

Mejor ser **consistente** con pocas normas...

- ... que cambiar de norma
- No me sorprendas
- No me hagas pensar

Recomendaciones - Bloques

> “El buen código es su mejor documentación”
> -- Steve McConnell

Objetivo:

- Reducir el número de líneas por bloque

Regla:

- Implementar e invocar una función para cada bloque
- Al menos si hay más de una instrucción

Recomendaciones - Funciones (I)

> “Una función debería hacer una sola cosa, hacerla bien, y hacerla solo ella”
> -- Ley de Curly

Pequeñas piezas para organizar programas

- Pequeñas y organizadas
- Cuanto más pequeñas más reutilizables
- DRY: Don’t Repeat Yourself
- Con verbos en su nombre que indiquen propósito
- ...sin comentarios

Recomendaciones - Funciones (y II)

> “Duplication is the primary enemy of a well-designed system”
> -- Robert C. Martin

- Cuantos menos argumentos mejor
- Mejor si son inmutables
- Mejor si son puras
- Un solo propósito
  - ... o al menos un mismo nivel de abstracción

Recomendaciones - Estructura de datos

"Data structures expose their data and have no meaningful functions"
-- Robert C. Martin

"Bad programmers worry about the code. Good programmers worry about data structures and their relationships"
-- Linus Torvalds

Sin comportamiento de negocio

Cohesionan variables relacionadas, objetos de negocio

Reducen argumentos en funciones



Recomendaciones - Objetos

> “La encapsulación es importante; pero la razón por la cual es importante es aún más importante. La encapsulación nos ayuda a razonar sobre nuestro código”
> -- Michael C. Feathers

Arquitectura limpia

Las clases, como las funciones: pequeñas y organizadas

Los objetos encapsulan la lógica

- Usan estructuras de datos
- Suelen representar Actores
- Relacionan unas entidades con otras
- Interfaces mejor que herencia



Índice

1. Clean Code
2. **Principios SOLID**
3. Clean architecture
4. Git y Git Flow

SOLID


S
SRP
Single
Responsability
Principle
O
OCP
Open /
Closed
Principle
L
LSP
Liskov
Substitution
Principle
I
ISP
Interface
Segregation
Principle
D
DIP
Dependency
Inversion
Principle

SO

SRP

- **Principio de responsabilidad única (Single responsibility principle)**
- Un objeto solo debería tener una única responsabilidad, o razón para cambiar

OCP

- **Principio de abierto/cerrado (Open/closed principle)**
- Las entidades de software deben estar abiertas para su extensión, pero cerradas para su modificación

LI

LSP

- **Principio de sustitución de Liskov (Liskov substitution principle)**
- Los objetos de un programa deberían ser reemplazables por instancias de sus subtipos sin alterar el correcto funcionamiento del programa

ISP

- **Principio de segregación de la interfaz (Interface segregation principle)**
- Muchas interfaces cliente específicas son mejores que una interfaz de propósito general

DIP

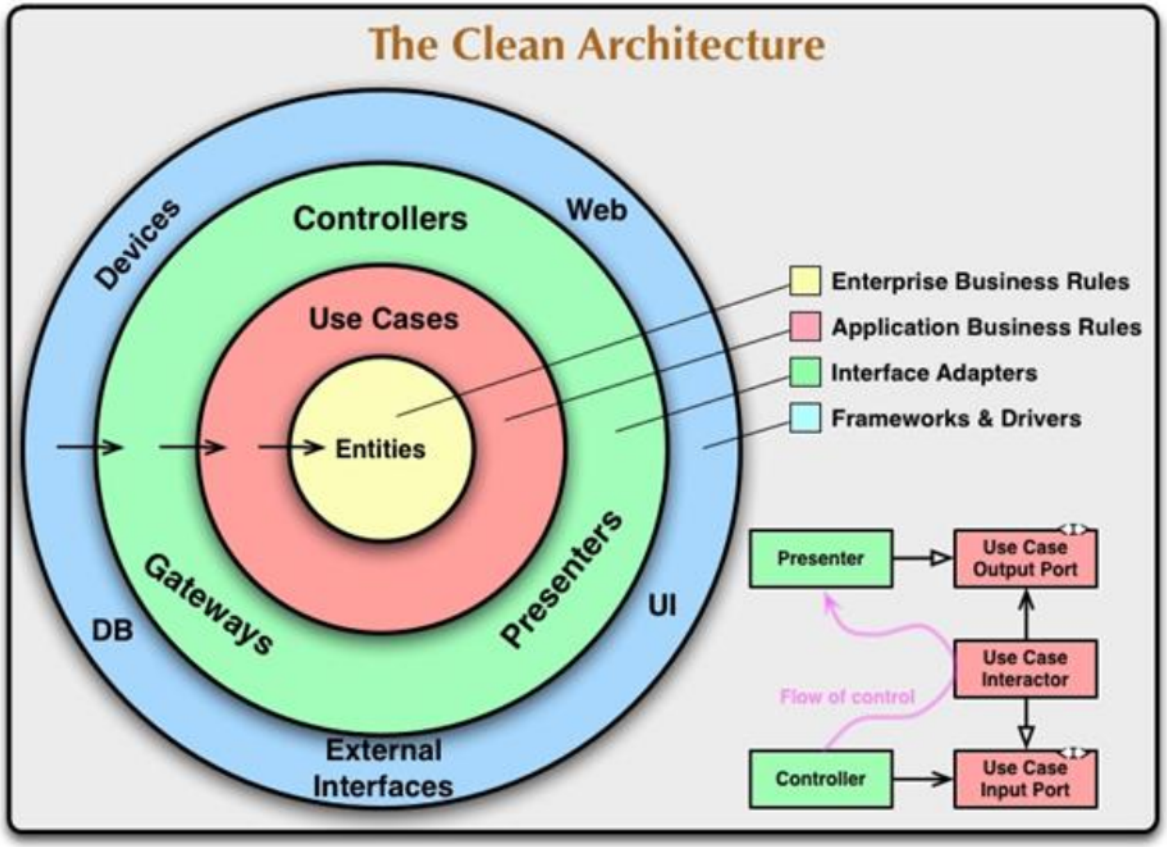
- **Principio de inversión de la dependencia (Dependency inversion principle)**
- Depender de abstracciones, no depender de implementaciones

"Objects hide their data behind abstractions and expose functions that operate on that data"
-- Robert C. Martin

Índice

1. Clean Code
2. Principios SOLID
3. **Clean architecture**
4. Git y Git Flow

CLEAN Architecture



Índice

1. Clean Code
2. Principios SOLID
3. Clean architecture
4. **Git y Git Flow**

GIT

- Software de control de versiones
  - Diseñado por Linus Torvalds
- Lleva registro de los cambios en archivos
- Permite coordinar el trabajo de varias personas sobre los mismos archivos
- Sistema distribuido
  - Funciona con múltiples repositorios
  - Cada usuario tiene su propio repositorio local
- Es de código libre bajo licencia GNU GPL v2



CONSEJO

UTILIZAD UN SISTEMA DE CONTROL DE VERSIONES

¡SIEMPRE!

CONSEJO

UTILIZAD UN SISTEMA DE CONTROL DE VERSIONES

¡SIEMPRE!

BUENO… EXCEPTO...

CONSEJO

UTILIZAD UN SISTEMA DE CONTROL DE VERSIONES

¡SIEMPRE!

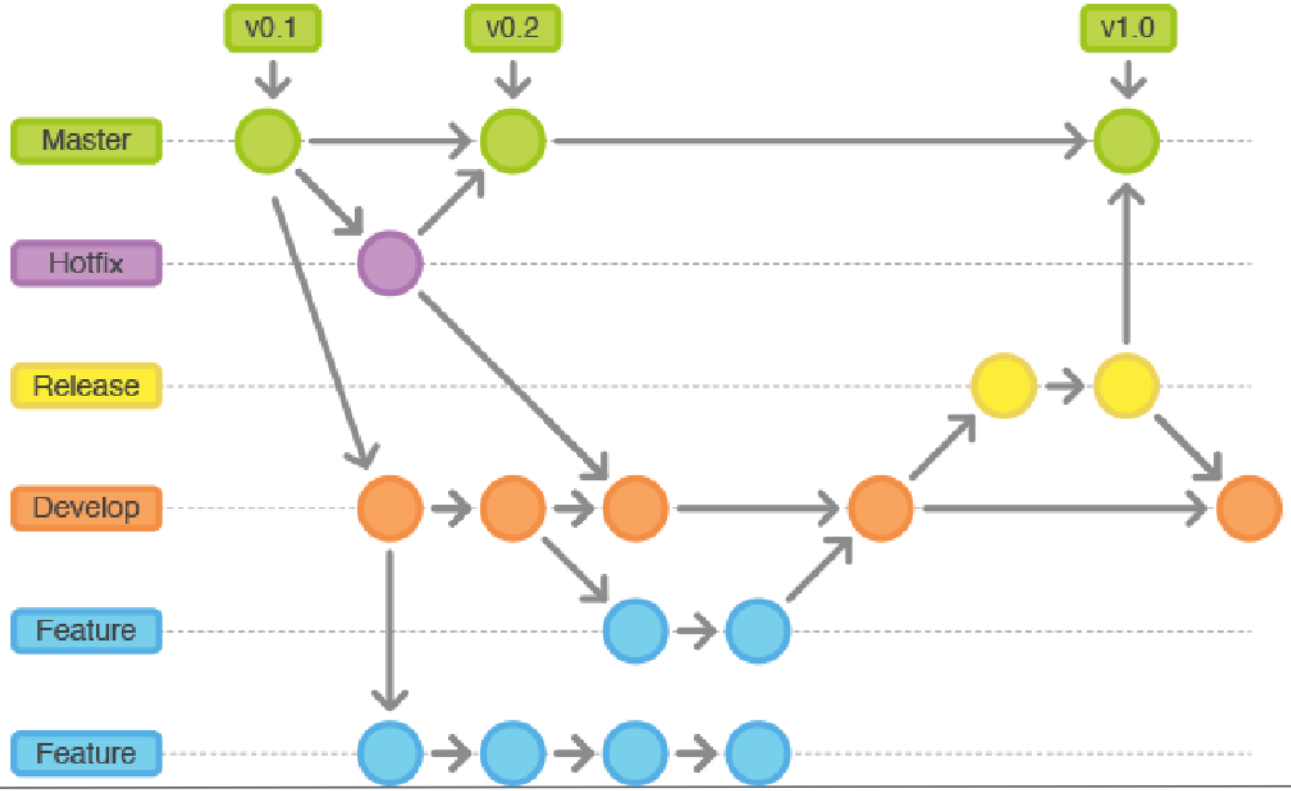
BUENO… EXCEPTO…

NO, ES BROMA, **¡SIEMPRE!**

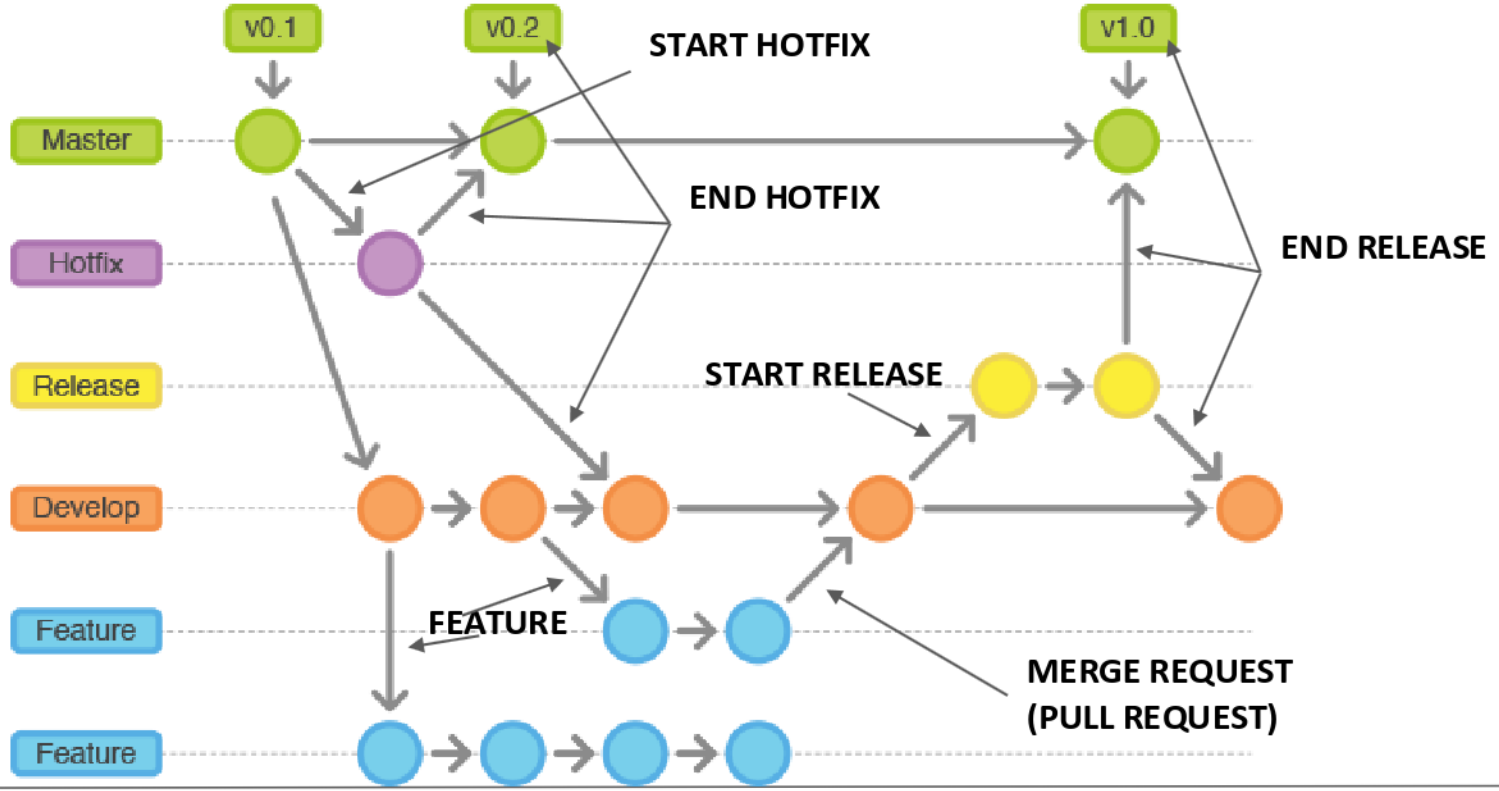
Branches and Tags

- Ramas o branches
  - Se trabaja en ramas que luego se juntan
  - Ramas comunes: *master*, *develop*, *feature*, *release*, *hotfix*
- Etiquetas o tags
  - Identifican un commit relevante
  - Normalmente indican entregas (o releases)
    - 1.0.0-RC.1; 2.1.0; 4.1.1-beta.3; ...

Git Flow



Git Flow


v0.1
v0.2
START HOTFIX
v1.0
Master
END HOTFIX
Hotfix
END RELEASE
Release
START RELEASE
Develop
Feature
FEATURE
MERGE REQUEST
(PULL REQUEST)
Feature

Software de GIT

- Repositorios remotos
  - GitHub: github.com
  - GitLab: gitlab.com
  - Bitbucket: bitbucket.org
- Aplicaciones de escritorio
  - SourceTree: sourcetreeapp.com
  - GitKraken: gitkraken.com
  - Plugin para Android Studio: https://github.com/OpherV/gitflow4idea/
- Consola de comandos

